



CSS



JS

RE AD ME

Style global

Renan Ramlov

1. A Fundação (Reset)

Antes de começar a decorar, precisamos de um terreno plano. O nosso sistema já vem com um "reset" que remove margens e preenchimentos indesejados, garantindo que tudo comece do zero. Além disso, a fonte oficial é a Inter, que traz clareza e modernidade para todo o texto.

2. Classes Utilitárias

Se precisa alinhar elementos use as nossas classes de Flexbox:

- .flex: Ativa o modo de organização flexível.
- .flex-col: Empilha os itens um em cima do outro, como uma torre.
- .flex-center: Coloca tudo exatamente no meio da tela.
- .flex-between: Empurra os itens para as pontas, deixando um espaço elegante entre eles.
- Espaçamento: Use os "gaps" (.gap-10, .gap-15, .gap-20) para dar um respiro entre os elementos sem complicação.

3. Ícones

Para usar um, você usa a sintaxe

<i class=" ícone cor Tamanho tipo"></i>

cor	vm(vermelho)	vd(verde)	bk(preto)	wt(branco)
tamanho	p(12px)	m(24px)	g(32px)	gg(48px)

T
i
p
o
s


IFC


Adicionar Documento


Baixar


Buscar


Calendário


Check


Documento


Editar


Início


Lista


Saída


Seta Baixo


Sobre


Usuário


Excluir


<i class=" ícone vd m IFC"></i>

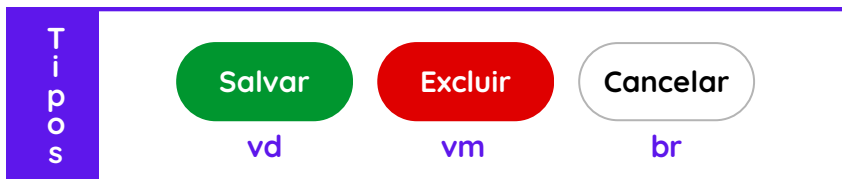

<i class=" ícone vm m excluir"></i>

4. Componentes de UI

Agora, vamos montar as partes interativas:

- Painéis: Use a classe `.painel` para criar áreas contidas de conteúdo, com um cabeçalho elegante usando `.painel-cabecalho`.
- Botões: Crie botões usando a sintaxe:

```
<button class="btn tipo">
  texto
</button>
```



- Formulários: Organize as perguntas com a classe `.campo`. Se o usuário cometer um erro, a classe `.input-erro` pintará o campo de vermelho para avisá-lo.

```
<div class="datas">
```

```
  <div class="campo">
```

```
    <label>Data Inicial</label>
```

```
    <input type="date" id="dataInicial">
```

```
  </div>
```

```
<div class="campo">
```

```
  <label>Data Final</label>
```

```
  <input type="date" id="dataFinal"
```

```
    onchange="validarDatas()">
```

```
    <small id="erroData"
```

```
      class="erro"></small>
```

```
  </div>
```

Script

para utilização do `.erro` e `.input-erro`, utilize um script

```
<script>
```

```
  function validarDatas() {
```

```
    dataFinal.classList.remove("input-erro");
```

```
    erroData.textContent = "";
```

```
    if (dataFinal.value <= dataInicial.value) {
```

```
      erroData.textContent = "Data inválida.";
```

```
      dataFinal.classList.add("input-
```

```
erro");
```

```
    }
```

```
  }
```

```
</script>
```



```
</div>
```

5. Organizando a Informação (Tabelas e Status)

Para listar dados de forma profissional:

- Use a `.tabela` dentro de um `.tabela-container` para que ela se comporte bem até em telas pequenas.
 - Status de Semáforo: Use as pílulas de status para indicar o momento de cada item:

Ativo

Ativo

Pendente

Pendente

Rejeitado

Rejeitado

6. Margens (Margins)

Estas classes são utilizadas para criar espaço fora dos elementos, empurrando os componentes vizinhos.

classe	ação	espaçamento
.mt-10	Margem Superior (Margin Top)	10px
.mt-15	Margem Superior (Margin Top)	15px
.mt-20	Margem Superior (Margin Top)	20px
.mb-10	Margem Inferior (Margin Bottom)	10px
.mb-15	Margem Inferior (Margin Bottom)	15px
.mb-20	Margem Inferior (Margin Bottom)	20px

7. Preenchimentos (Paddings)

Estas classes definem o espaço interno de um elemento, entre o conteúdo e a borda

classe	ação	Espaçamento (Todos os lados)
.p-10	Preenchimento Interno	10px
.p-15	Preenchimento Interno	15px
.p-20	Preenchimento Interno	20px

8. Espaçamentos de Grade (Gaps)

Utilizados em conjunto com as classes `.flex`, estes utilitários definem a distância entre elementos filhos.

classe	ação	Espaçamento
.gap-10	Espaço entre itens	10px
.gap-15	Espaço entre itens	15px
.gap-20	Espaço entre itens	20px

Resumo Rápido:

- **Margem:** Empurra o que está fora.
- **Preenchimento (Padding):** Afasta o conteúdo de dentro.
- **Gap:** Define a distância entre os itens de um grupo